

Session Management Specification for *Ragtime-Pro* – *Static RAG + Hybrid Retrieval + Cross-Encoder + R2 Memory*

1. Purpose

The session management subsystem provides:

- A **unique session identifier** for each conversation.
- A mechanism to **persist conversation history** in Cloudflare R2.
- A **fingerprint-based continuity heuristic** to detect returning users without accounts.
- A safe **user confirmation flow** before restoring previous sessions.
- A deterministic interface for reading/writing session data from Vercel serverless functions.

This system enables multi-turn interactions while maintaining full statelessness at runtime.

2. Session Identifier

2.1 Primary Identifier (Canonical)

Each conversation is uniquely identified by a **UUID v4**:

Code

```
sessionId = crypto.randomUUID()
```

This ID is:

- Random
- Collision-proof
- Opaque
- Not derived from user data

2.2 Storage on Client

The session ID is stored in a secure HTTP cookie:

Code

Name: rag_session

Value: <sessionId>

HttpOnly: true

SameSite: Lax

Path: /api

Max-Age: 30 days

This cookie persists across browser restarts and page reloads.

3. Fingerprint Heuristic

3.1 Purpose

Used only when:

- The user has **no session cookie**, and
- We want to detect whether they may have a previous session.

3.2 Fingerprint Definition

Code

```
fingerprint = sha256(ip + userAgent)
```

Where:

- ip = client IP from request headers
- userAgent = browser user-agent string

3.3 Privacy Requirements

- Never store raw IPs.
- Only store the hashed fingerprint.
- Only use fingerprint for **possible matches**, never as identity.

4. Session File Format (R2)

Stored under:

Code

```
conversations/{sessionId}.json
```

Structure:

json

```
{
```

```
  "sessionId": "uuid",
```

```
  "fingerprint": "hashed-ip-ua",
```

```
"history": [  
  { "role": "user", "content": "..."},  
  { "role": "assistant", "content": "..."}  
],  
"summary": "Optional running summary of conversation.",  
"lastSeen": "2026-08-13T15:18:00Z"  
}
```

Fields

Field	Description
sessionId	Canonical session identifier
fingerprint	Hashed IP + UA for continuity detection
history	Full list of messages (trimmed as needed)
summary	Optional compressed history for long conversations
lastSeen	ISO timestamp of last interaction

5. Session Lifecycle

5.1 New Request Flow

When a request arrives:

1. **Check for cookie** `rag_session`
 - If present → load session from R2.
 - If missing → proceed to fingerprint matching.
2. **Compute fingerprint**

Code

```
fingerprint = sha256(ip + userAgent)
```

3. **Search R2 for matching fingerprint**
 - If none found → create new session.
 - If found → proceed to confirmation.
4. **Ask user for confirmation**

Code

“It looks like you may have a previous conversation.

Would you like to continue where you left off?”

5. User response

- If **yes** → load previous session.
- If **no** → create new session.

6. Creating a New Session

When creating a new session:

1. Generate sessionId = crypto.randomUUID()
2. Create initial session file:

json

```
{  
  "sessionId": "<uuid>",  
  "fingerprint": "<hashed-ip-ua>",  
  "history": [],  
  "summary": "",  
  "lastSeen": "<timestamp>"  
}
```

3. Save to R2 under:

Code

conversations/<sessionId>.json

4. Set cookie rag_session=<sessionId>

7. Updating a Session

On each turn:

1. Load session file from R2.
2. Append new user message to history.
3. Append assistant message after response generation.
4. If history.length exceeds threshold (e.g., 20 messages):

- Generate/update summary using Azure OpenAI.
 - Truncate older messages.
5. Update lastSeen.
 6. Write updated file back to R2.

8. Session Retrieval API (Claude must implement)

8.1 readSession(sessionId)

- Fetches conversations/{sessionId}.json
- Returns parsed JSON
- If missing → return null

8.2 writeSession(sessionData)

- Writes JSON to R2
- Overwrites existing file

8.3 findByFingerprint(fingerprint)

- Lists objects under conversations/
- Returns the most recent session with matching fingerprint
- If none → return null

8.4 createSession(ip, userAgent)

- Generates new sessionId
- Computes fingerprint
- Creates new session file
- Returns sessionId + session object

8.5 updateSession(sessionId, updates)

- Merges updates into existing session
- Writes back to R2

9. Error Handling

Missing Session File

If readSession returns null:

- Create a new session

- Set cookie
- Continue normally

Corrupted JSON

- Log error
- Create new session
- Do not attempt recovery

Fingerprint Collision

If multiple sessions match:

- Select the most recent (lastSeen)
- Ask user for confirmation

10. Security Requirements

- All R2 operations must use **private API tokens** stored in Vercel environment variables.
- Session cookies must be:
 - HttpOnly
 - SameSite=Lax
 - Not accessible to client JS
- Fingerprints must be hashed (SHA-256).
- No raw IPs stored anywhere.

11. Integration Points

Used by:

- Retrieval pipeline (to inject conversation history)
- Answer generation (to include summary + recent turns)
- Frontend chat UI (to maintain continuity)

Provides:

- Deterministic session identity
- Persistent multi-turn memory
- Safe continuity detection

- Clean R2 storage structure